

NKUA – DIT | K29 – DESIGN AND USE OF DATABASES

Project 1: Hydroplane Company Database

Dimitrios Kostinis (ID 1112 2021 00235)

Introduction

The present report documents a conceptual design of the database for the airline company «K29-HYDRO». The main goal of my modeling approach was the development of a robust and scalable system, capable of handling the complex operational needs and demands of this company concerning the organization of the specialized personnel, the company's fleet, flight scheduling, customer data, destinations and many more. I achieved this through the strategic use of the design tools given to us by the E/R modeling framework from this class's related material. In the following chapters I attempt to analyze and justify my choices in the dilemmas I encountered and my whole troubleshooting process. Finally, the report also includes some business logical constraints that, due to their nature, cannot be accounted for by the diagram alone.

Chapter 1: Employee entity set

It is apparent entity "Employees" is a broader term for a category of professionals, each with their own tasks, expertise and attributes that differentiates them. Thus, I choose to break down the superclass "Employees" into the subclasses (entities) "Pilots", "Assistants", "Technicians" and "Admins" using an ISA hierarchy. Another variable we should consider is the dependents for insurance that each employee has. They will be portrayed by weak entity with thick lines since everyone in the family must be insured. The relationship "Policy" between "Employees" and "Dependents" is 1:N (1 employee can have 0 or more dependents). Lastly, each employee reports to the manager of his department, as is seen by the "Reports to" self-relationship.

Chapter 2: Planes entity set

On first thought an ISA hierarchy again could be taken advantage of since we have a superclass "plane" and three subclasses for the three types. However, this choice could lead to issues and inefficiencies in our database design. Just to name a few, more tables would be required, thus longer traversal time (when asking for a query for mileage for example). Also, if the company bought a different type of plane the structure of the database will have to be altered (new sub-entity) instead of just adding another value to the existing entity "plane type". Lastly, all three have the exact same attributes of interest. For those reasons, a 1:N relationship (many airplanes of one type) will be used with the name "is type" with the thick line to indicate full participation. Also, the entity "plane type" can't be an attribute since we must consider the pilots and technicians that specialize to specific planes and we can't connect entities with attributes of a different entity. Given attributes of "planes" are name and kilometer count and for "plane types" class, capacity and speed.

Chapter 3: Entity set Geography

By the company's profile report we can pinpoint the need for these three, separate, entities "Waterports", "Regions" and "Destinations". Each waterport is tied to one region and vice versa (1:1 relationship). Now we should examine the use of binary versus ternary relationships. In my judgment, the use of binary relationships in the given order provides all the necessary information for every query. Using a more complex ternary relationship isn't needed since each entity doesn't need all the others for context (eg waterports aren't needed to describe the destinations). We will use this relationship "serves" between waterports and regions and "belongs to" between regions and destinations. We have total participation on all fronts.

A moment to look back before constructing more entities...

At this point we have built the operational framework for our operation. Before starting to consider the customers, ratings and schedules aspects of it, it is sound to examine the relationships between our existing entity sets. As dictated, every pilot and technician (full participation I hope, otherwise layoffs are out of my pay grade) specialize in one or more types of planes. In the beginning I listed those constraints as attributes, however now I see they can work as N:M relationships "certified to fly" and "maintains" to connect those entities, as it is needed. Also, between the entity sets "Planes" and "Geography" we should add the relationship "base location" as each plane has a waterport as a base (N:1 relationship).

Chapter 4: Customers, Booking system and Financials

Moving on to the booking process, my initial thought was to create the entity "dependent customers", a weak entity tied directly on the entity "Customers". "Customers" is set to store in the database all the necessary information regarding passenger's full name, date of birth, phone number, email and password. However, a scenario came to mind: what if a customer makes more than one reservation in the future? Could the DBMS figure out the specific flight the dependent is joining? Logically, in this situation the answer is not very clear. To resolve this, I created a new, strong, entity called "Reservations" that connects to both flight and the customer, 1:N relationship with customers (one customer can create many reservations) and N:1 relationship with the flights (many reservations for one flight). Consequently, the dependent customer is now a weak entity depending strictly on the reservation they belong to. For the financial part and with respect to the company's requirements I added the "Transaction" entity connected to the reservation. The accumulation of values in this domain automatically forms the required history log. Also present are essential transactional attributes transaction ID, amount and date.

Chapter 5: Flights and Supervision

One of the most complex design dilemmas I encountered was connecting the planes, the flights and the ground personnel. "Flight" is the entity in which flight data is to be stored, attribute and primary key is "flight id". At first glance, a ternary relationship linking all three seemed like a good choice. However, then I realized this kind of relationship implies that all three entities must exist simultaneously which is not necessary by the given guidelines. The assignment of a plane to a flight is an independent scheduling event that exists before a technician is even assigned to inspect it. For this reason, I decided to use the tool of aggregation, a well-fitting adjustment since we don't have to alter the flight and assigned plane relationship, only set a

technician to inspect it. This aggregation is achieved through the relationship monitors. Also concerning flights, I connected them with waterports with a 1:1 relationship since passengers can have a connecting flight to one of the three waterports before reaching their destinations. Furthermore, flights are connected to destinations with a 1:1 relationship, as each flight has a specific endpoint. As for the flight crew (pilots and assistants) they are simply linked directly to the flight entity with the relationships “pilot assignment” and “assistants assignment”.

Chapter 6: Customer Reviews and Loyalty packages

Concerning flyer clubs, the company features three tiers (Basic, Silver and Platinum), each with their own discounts and perks. Instead of simply adding tier and discount as attributes to the customer, I chose to create a separate “club tier” entity connecting it to the customers via a 1:N relationship. The reason behind this was scalability: what if the company decided to add one another tier? If it was an attribute, we would have to alter the database schema of customers instead of just adding another value. Finally, the reviews were added between the entities “customers” and “flight” with 1:N and N:1 relationships “uploads” and “for flight” between them accordingly, so that one customer can post many reviews for one flight and many reviews can be for one flight. Attributes to reviews are score and comments.

Chapter 7: Business logical constraints

As requested by the project specifications certain operational rules cannot be effectively depicted through an ER diagram. Some of these include:

1. Certifications checking: before a pilot can be assigned to a flight, the system must verify that the pilot exists in the “certified for” table for the specific plane type for that route.
2. Capacity checking: before confirming a reservation, the system must ensure that the total number of passengers does not exceed the plane’s capacity attribute value.
3. Club tier upgrades: The system should monitor a customer’s kilometer count and automatically upgrade the customer’s tier when he exceeded the threshold.
4. Overlapping flights: system must check if a plane or a pilot has been signed on two different flights at the same time.
5. Discount implementation: when creating a transaction, system must check the club tier of the corresponding customer so it can implement the correct discount.